



PONTIFICIA UNIVERSIDAD JAVERIANA CALI

Procesamiento de Grandes Volúmenes de Datos

Ingeniería de Sistemas y Computación
Departamento de Electrónica y Ciencias de la Computación

Parcial 2 - Data Warehouse & Data Lake

Fecha y Hora de entrega: miércoles 7 de noviembre a las 2 pm.

Entregas posteriores a la fecha y hora tendrán una penalización de -1 punto a la nota final

Enviar solución al correo: oscar.ceballos@javerianacali.edu.co

Objetivo:

Implementar una solución integral de ingeniería de datos y business intelligence que permita la ingesta, integración, modelado dimensional y explotación analítica de información deportiva heterogénea para optimizar la toma de decisiones basada en el rendimiento de los atletas.

Integantes:

El parcial se podrá realizar de manera individual o en grupos de máximo 3 estudiantes.

Entregables:

El único entregable de código requerido por parte del estudiante es el script en Python/PySpark (extensión .ipynb) encargado de extraer, limpiar, integrar y cargar la información hacia el Data Warehouse. Cada estudiante o grupo deberá enviar al correo oscar.ceballos@javerianacali.edu.co un único archivo comprimido en formato .zip nombrados con la siguiente estructura: `Parcial_1_PySpark_Apellido_Nombre.ipynb` y `Parcial_1_PySpark_Apellido_Nombre.zip`, respectivamente. Estudiantes o grupos de estudiantes que no cumplan con el formato se les aplicará una penalización de -0.5 puntos a la nota final



PONTIFICIA UNIVERSIDAD JAVERIANA CALI

Procesamiento de Grandes Volúmenes de Datos

Ingeniería de Sistemas y Computación

Departamento de Electrónica y Ciencias de la Computación

Proyecto de Desarrollo:

Data Mart de Rendimiento de Atletas (dwh_player_performance)

1. Contexto de Negocio y Caso de Uso

Contexto

Las organizaciones deportivas y de gestión de eventos necesitan consolidar información fragmentada para medir el impacto, rendimiento y comportamiento de los atletas en diferentes competencias. Actualmente, la base de datos transaccional (db_sports) almacena datos operativos detallados, pero carece de un modelo optimizado para la analítica de negocio.

Caso de Uso

Implementar un modelo dimensional en estrella (dwh_player_performance) que integre fuentes de datos heterogéneas (datos relacionales transaccionales y archivos planos externos con información de contactos y membresías). Esto permite a los analistas de negocio y directores deportivos evaluar el rendimiento de los atletas, segmentar por niveles de membresía y género, y analizar el impacto del estado de los eventos en las puntuaciones mediante consultas de Inteligencia de Negocio avanzadas.

2. Insumos

Para que el estudiante pueda enfocar su esfuerzo de desarrollo exclusivamente en la construcción del script de Python (ETL), se han preparado los siguientes elementos como insumos oficiales del proyecto. Estos artefactos ya se encuentran definidos y estructurados para ser consumidos o desplegados antes de ejecutar el código.

Insumo 1: Script DDL del Data Mart

El script SQL define la estructura de destino en dwh_sports donde el script de Python deberá volcar los datos para poblar las dimensiones y la tabla de hechos procesadas. El estudiante puede ejecutarlo previamente en PostgreSQL para preparar el entorno de almacenamiento.

SQL

-- Creación de la dimensión de Eventos

```
CREATE TABLE dim_events (  
    id integer NOT NULL,  
    event_key varchar(100) NOT NULL,  
    event_status varchar(50),  
    start_date timestamp,  
    PRIMARY KEY (id)  
);
```

-- Creación de la dimensión de Posiciones

```
CREATE TABLE dim_positions (  
    id integer NOT NULL,  
    abbreviation varchar(50) NOT NULL,  
    PRIMARY KEY (id)  
);
```

-- Creación de la dimensión de Personas enriquecida (SCD / Multifuente)

```
CREATE TABLE dim_persons_performance (  
    id integer NOT NULL,  
    entity_id integer NOT NULL,  
    first_name varchar(50) NOT NULL DEFAULT 'N/A',  
    last_name varchar(50) NOT NULL DEFAULT 'N/A',  
    gender varchar(10),  
    email varchar(100), -- Dato proveniente de CSV externo 1  
    membership_level varchar(50), -- Dato proveniente de CSV externo 2  
    version integer DEFAULT 1,  
    date_from timestamp,  
    date_to timestamp,  
    PRIMARY KEY (id)  
);  
  
CREATE INDEX idx_dim_persons_perf_entity ON dim_persons_performance(entity_id);
```

-- Creación de la Tabla de Hechos: Rendimiento de Participantes en Eventos

```
CREATE TABLE ft_player_performance (  
    dateid integer NOT NULL,
```



PONTIFICIA UNIVERSIDAD JAVERIANA CALI

Procesamiento de Grandes Volúmenes de Datos

Ingeniería de Sistemas y Computación

Departamento de Electrónica y Ciencias de la Computación

```
id_event integer NOT NULL,  
id_person integer NOT NULL,  
id_position integer NOT NULL,  
score_numeric numeric(10,2) DEFAULT 0.00,  
rank integer DEFAULT 0,  
PRIMARY KEY (dateid, id_event, id_person)  
);
```

Insumo 2: Archivos .csv cargados en el Data Lake

Los siguientes archivos representan la fuente de datos externa, almacenados en el bucket **rawdata** del Data Lake (MinIO). Estos archivos deberán leerse desde el script de Python, el cual debe leerlos y limpiarlos para unirse con la base de datos operacional db_sports.

Dataset: person_contacts.csv

None

```
person_id,email  
1, john.doe@example.com  
2,sarah.smith@sportmail.org  
3,michael.jones@athlete.net  
4,emily.brown@sportsmail.com  
5,david.wilson@performance.io  
6,jessica.taylor@training.org  
7,daniel.anderson@fitnet.com  
8,laura.thomas@active.co  
9,james.jackson@runner.net  
10,olivia.white@olympus.org  
11,robert.harris@sportshub.com  
12,sophia.martin@athleteplus.io  
13,william.garcia@kinesis.org  
14,isabella.martinez@vitality.com  
15,charles.robinson@endurance.net  
16,mia.clark@motion.org  
17,thomas.rodriguez@speed.com  
18,charlotte.lewis@powerfit.io
```



PONTIFICIA UNIVERSIDAD JAVERIANA CALI

Procesamiento de Grandes Volúmenes de Datos

Ingeniería de Sistemas y Computación

Departamento de Electrónica y Ciencias de la Computación

19,christopher.lee@kinetic.org
20,amelia.walker@fitpro.com
21,matthew.hall@stamina.net
22,harper.allen@apex.org
23,anthony.young@athletezone.com
24,evelyn.hernandez@cardio.io
25,mark.king@velocity.org
26,abigail.wright@fitlife.com
27,steven.scott@ironman.net
28,emily.torres@peak.org
29,andrew.nguyen@prosport.com
30,elizabeth.hill@triathlon.io
31,joshua.flores@champion.org
32,sofia.green@gymnast.com
33,kevin.adams@fighter.net
34,camila.nelson@runner.org
35,brian.baker@crossfit.com
36,gianna.hall@athlete.io
37,edward.rivera@sport.org
38,aria.campbell@active.net
39,ronald.mitchell@fit.com
40,scarlett.carter@vital.io
41,timothy.roberts@power.org
42,victoria.gomez@swift.com
43,jason.phillips@motion.net
44,madison.evans@endurance.io
45,jeffrey.turner@speed.org
46,luna.diaz@kinesis.com
47,ryan.parker@peak.net
48,grace.cruz@stamina.org
49,gary.edwards@apex.com
50,chloe.collins@athletezone.io



PONTIFICIA UNIVERSIDAD JAVERIANA CALI

Procesamiento de Grandes Volúmenes de Datos

Ingeniería de Sistemas y Computación

Departamento de Electrónica y Ciencias de la Computación

Dataset: person_memberships.csv

None

person_id,membership_level

1,Gold

2,Silver

3,Platinum

4,Standard

5,Gold

6,Platinum

7,Standard

8,Silver

9,Gold

10,Platinum

11,Standard

12,Gold

13,Silver

14,Platinum

15,Standard

16,Gold

17,Silver

18,Platinum

19,Standard

20,Gold

21,Platinum

22,Standard

23,Silver

24,Gold

25,Platinum

26,Standard

27,Gold

28,Silver

29,Platinum

30,Standard

31,Gold

32,Platinum

33,Standard

34,Silver

35,Gold



PONTIFICIA UNIVERSIDAD JAVERIANA CALI

Procesamiento de Grandes Volúmenes de Datos

Ingeniería de Sistemas y Computación
Departamento de Electrónica y Ciencias de la Computación

36,Platinum
37,Standard
38,Gold
39,Silver
40,Platinum
41,Standard
42,Gold
43,Silver
44,Platinum
45,Standard
46,Gold
47,Platinum
48,Standard
49,Silver
50,Gold

Insumo 3: Base de Datos Transaccional db_sports

Base de datos relacional operativa contiene el registro transaccional y operativo de la actividad deportiva. El estudiante deberá utilizar las siguientes entidades o tablas de db_sports:

1. events:

- Propósito: Almacenar los datos generales de los eventos deportivos.
- Campos clave: id, event_key, event_status, start_date_time.

2. positions:

- Propósito: Contener los roles de los participantes dentro de las disciplinas.
- Campos clave: id, affiliation_id, abbreviation.

3. persons:

- Propósito: Almacenar la demografía básica y metadatos de los individuos.
- Campos clave: id (mapeado como entity_id), gender.

4. display_names:

- Propósito: Registrar la información de las entidades.
- Campos clave: entity_id, first_name, last_name.

5. participants_events:



PONTIFICIA UNIVERSIDAD JAVERIANA CALI

Procesamiento de Grandes Volúmenes de Datos

Ingeniería de Sistemas y Computación

Departamento de Electrónica y Ciencias de la Computación

- Propósito: Tabla puente que vincula a los participantes con los eventos deportivos, registrando su participación y métricas operativas iniciales.
- Campos clave: participant_id, event_id, score, rank

Insumo 4: Plantilla base para el desarrollo de la solución

Script en blanco con comentarios guía y secciones marcadas con # TODO

Python

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.window import Window

# =====
# 0. INICIALIZACIÓN DE LA SESIÓN DE SPARK
# =====

spark = (SparkSession.builder
    .appName("ETL-DataMart-PlayerPerformance")
    .master("local[*]")
    .config("spark.driver.memory", "1g")
    .config("spark.executor.memory", "1500m")
    .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension")
    .config("spark.sql.catalog.spark_catalog",
"org.apache.spark.sql.delta.catalog.DeltaCatalog")
    .config("spark.hadoop.fs.s3a.endpoint", "http://localhost:9000")
    .config("spark.hadoop.fs.s3a.access.key", "minioadmin")
    .config("spark.hadoop.fs.s3a.secret.key", "minioadmin123")
    .config("spark.hadoop.fs.s3a.path.style.access", "true")
    .config("spark.hadoop.fs.s3a.impl", "org.apache.hadoop.fs.s3a.S3AFileSystem")
    .config("spark.hadoop.fs.s3a.connection.ssl.enabled", "false")
    .config("spark.hadoop.fs.s3a.aws.credentials.provider",
"org.apache.hadoop.fs.s3a.SimpleAWSCredentialsProvider")
    .getOrCreate())

jdbc_url = "jdbc:postgresql://localhost:5432/db_sports"
dw_url = "jdbc:postgresql://localhost:5432/dwh_sports"
```



PONTIFICIA UNIVERSIDAD JAVERIANA CALI

Procesamiento de Grandes Volúmenes de Datos

Ingeniería de Sistemas y Computación

Departamento de Electrónica y Ciencias de la Computación

```
properties = {"user": "postgres", "password": "abc123", "driver":  
"org.postgresql.Driver"}
```

```
# =====
```

```
# 1. CARGA Y TRANSFORMACIÓN DE DIMENSIONES SECUNDARIAS (15%)
```

```
# =====
```

```
# 1.1 Dimensión de Eventos (`dim_events`)
```

```
# TODO: Extraer la tabla 'events', renombrar 'id' a 'entity_event_id',
```

```
# generar una llave subrogada incremental 'id' con Window y escribir en el DW
```

```
# 1.2 Dimensión de Posiciones (`dim_positions`)
```

```
# TODO: Extraer la tabla 'positions', filtrar nulos en 'abbreviation',
```

```
# obtener valores únicas, generar llave subrogada incremental y escribir en el DW
```

```
# =====
```

```
# 2. ETL MULTIFUENTE PARA `dim_persons_performance` (20%)
```

```
# =====
```

```
# TODO:
```

```
# - Extraer 'persons' y 'display_names' de la BD operacional.
```

```
# - Ingestar 'person_contacts.csv' y 'person_memberships.csv' desde el Data Lake  
(s3a://rawdata/).
```

```
# - Limpiar y estandarizar textos (lower, trim, coalesce).
```

```
# - Ejecutar LEFT JOINS sucesivos para integrar las fuentes.
```

```
# - Asignar llave subrogada y metadatos de control (version, date_from) y guardar  
en el DW.
```

```
# =====
```

```
# 3. CARGA DE LA TABLA DE HECHOS `ft_player_performance` (35%)
```

```
# =====
```

```
# TODO:
```

```
# - Leer las dimensiones ya guardadas en el DW (`dim_events`,  
`dim_persons_performance`, `dim_positions`).
```

```
# - Extraer 'participants_events' y cruzar con eventos y posiciones operacionales.
```

```
# - Resolver las llaves foráneas haciendo JOINS con las dimensiones del DW.
```



PONTIFICIA UNIVERSIDAD JAVERIANA CALI

Procesamiento de Grandes Volúmenes de Datos

Ingeniería de Sistemas y Computación
Departamento de Electrónica y Ciencias de la Computación

```
# - Formatear la métrica 'dateid' (YYYYMMDD), castear 'score' a numérico y  
eliminar duplicados.  
# - Escribir el resultado final en la tabla 'ft_player_performance' del DW.
```

3. Especificación Técnica del Proceso ETL y Transformaciones

El pipeline ETL deberá diseñarse para ejecutarse de manera idempotente, garantizando la consistencia y la limpieza de los datos antes de cargarlos al Data Mart.

1. Carga y Transformación de Dimensiones Secundarias

1.1 Dimensión de Eventos (dim_events)

- Extracción: Leer la tabla transaccional *events* desde *db_sports*.
- Transformaciones:
 1. Renombrar la columna operacional *id* a *entity_event_id* para preservar la trazabilidad de origen.
 2. Seleccionar de forma única (*.distinct()*) los campos *event_key*, *event_status* y *start_date_time* (renombrado a *start_date*).
 3. Crear una ventana analítica ordenada por *entity_event_id* (*Window.orderBy("entity_event_id")*).
 4. Generar una nueva columna autoincremental llamada *id* utilizando la función de ventana *F.row_number()*, que actuará como la llave subrogada del Data Mart.
- Carga: Guardar el resultado en la tabla *dim_events* del esquema *dwh_sports* utilizando el modo de escritura *overwrite*.

1.2 Dimensión de Posiciones (dim_positions)

- Extracción: Leer la tabla transaccional *positions* desde *db_sports*.
- Transformaciones:

1. Seleccionar la columna *abbreviation*, filtrando y descartando explícitamente los valores nulos (`isNotNull()`).
 2. Obtener un listado de abreviaturas únicas (`.distinct()`).
 3. Definir una ventana de ordenamiento basada en la abreviatura (`Window.orderBy("abbreviation")`).
 4. Generar la llave subrogada incremental id mediante `F.row_number()`.
- Carga: Escribir el DataFrame resultante en la tabla *dim_positions* de *dwh_sports* en modo `overwrite`.

2. ETL Multifuente para la Dimensión de Personas (*dim_persons_performance*)

- Extracción de Origen (BD Operacional): Leer las tablas *persons* (campos *id* como *entity_id*, *gender*) y *display_names* (campos *entity_id*, *first_name*, *last_name*).
- Extracción de Origen (Data Lake / S3A): Leer los archivos CSV desde la capa *raw* (*s3a://rawdata/*) con inferencia de esquema y cabecera activada:
 1. *person_contacts.csv* (contiene *person_id*, *email*).
 2. *person_memberships.csv* (contiene *person_id*, *membership_level*).
- Reglas de Limpieza y Estandarización:
 1. *Datos Operacionales*: Hacer un LEFT JOIN entre *persons* y *display_names*. Aplicar `coalesce` para rellenar nulos en nombres y apellidos con la cadena "N/A". Estandarizar el género a minúsculas eliminando espacios sobrantes (`F.lower(F.trim(...))`).
 2. *Archivo de Contactos*: Convertir los correos electrónicos a minúsculas y limpiar espacios (`F.lower(F.trim(...))`).
 3. *Archivo de Membresías*: Asegurar el formato capitalizado de los niveles (`F.initcap(F.trim(...))`).
- Integración y Enriquecimiento (Joins):
 1. Cruzar los datos limpios de personas con los contactos y membresías mediante sendos LEFT JOIN usando el identificador de persona (*entity_id* == *person_id*).
 2. Aplicar `coalesce` posteriores para asegurar que si un usuario no posee datos de contacto o membresía externos, se asignen valores por defecto ("no-email@domain.com" y "Standard", respectivamente).

- Generación de Metadatos de Control Dimensional:
 1. Crear una clave subrogada incremental id ordenada por entity_id.
 2. Añadir atributos de control de tipo Slowly Changing Dimension (SCD): version (entero inicializado en 1), date_from (marca de tiempo actual con F.current_timestamp()), y date_to (nulo por defecto).
- Carga: Almacenar la dimensión enriquecida en la tabla dim_persons_performance de dwh_sports con el modo overwrite.

3. Carga de la Tabla de Hechos (ft_player_performance)

- Lectura de Dimensiones del DW: Cargar desde el Data Warehouse las dimensiones ya procesadas (dim_events, dim_persons_performance, dim_positions) para recuperar las llaves subrogadas técnicas.
- Extracción Operacional: Leer las tablas operacionales participants_events (para métricas de puntaje y rango) y positions (para asociar roles) y events (para alinear claves de eventos).
- Resolución de Integridad Referencial y Joins:
 1. Relacionar la tabla de participación con la posición correspondiente asegurando el cruce correcto por identificador de afiliación/participante.
 2. Cruzar con la tabla operativa de eventos para alinear las claves de negocio (event_key).
 3. Realizar INNER JOIN y LEFT JOIN contra las dimensiones del Data Warehouse (dim_events, dim_persons_performance, dim_positions) utilizando las claves lógicas correspondientes para mapear las llaves sustitutas (id_event, id_person, id_position).
- Transformación y Cálculo de Métricas:
 1. Construir la llave de fecha dateid en formato numérico YYYYMMDD a partir de la fecha de inicio del evento (respaldada por la fecha actual si llegase a existir un nulo).
 2. Castear la métrica de puntaje a tipo numérico con manejo de nulos por defecto en 0.00 (score_numeric).
 3. Manejar los nulos del ranking operativo inicializándolos en 0.

4. Aplicar un filtro de deduplicación estricto sobre la clave primaria compuesta (dateid, id_event, id_person) para evitar violaciones de integridad referencial.
- Carga Final: Escribir el DataFrame consolidado en la tabla ft_player_performance de dwh_sports en modo overwrite (o append según la estrategia de particionamiento iterativo).

4. Catálogo de Consultas Analíticas

Para validar el éxito del proyecto, el estudiante debe ejecutar las siguientes consultas:

(3%) Consulta 1: Identificar el Top 10 de atletas con mayor puntuación promedio. Para ello, se debe calcular el puntaje promedio obtenido por cada atleta a lo largo de sus participaciones, considerando únicamente a aquellos que cuentan con un nivel de membresía registrado. El resultado se ordena de mayor a menor promedio, permitiendo a los directores deportivos o gestores de la plataforma reconocer rápidamente a los atletas de élite o de mayor rendimiento histórico.

SQL

SELECT

-- TODO: Seleccionar nombres, apellido y membresía de la dimensión de personas

-- TODO: Contar el total de eventos participados desde los hechos

-- TODO: Calcular el promedio de 'score_numeric' con ROUND y casting a ::numeric

FROM ft_player_performance f

JOIN dim_persons_performance p ON -- TODO: Cruzar por llave de persona

GROUP BY -- TODO: Agrupar por los campos de persona seleccionados

ORDER BY -- TODO: Ordenar descendientemente por el promedio obtenido

LIMIT 10;

(5%) Consulta 2: Analizar el rendimiento segmentado por nivel de membresía y género. Agrupar a los participantes cruzando su categoría de membresía (como Gold, Silver, Platinum, etc.) con su género, para calcular cuántos atletas únicos participan, cuántas participaciones totales acumulan, cuál es su puntaje promedio general y cuál ha sido la puntuación máxima



PONTIFICIA UNIVERSIDAD JAVERIANA CALI

Procesamiento de Grandes Volúmenes de Datos

Ingeniería de Sistemas y Computación

Departamento de Electrónica y Ciencias de la Computación

alcanzada en cada segmento. Es ideal para entender qué tipo de membresía concentra a los deportistas más activos o con mejores resultados cuantitativos.

SQL

SELECT

```
-- TODO: Seleccionar nivel de membresía y género de la dimensión de personas
-- TODO: Contar participantes únicos (DISTINCT) desde los hechos
-- TODO: Contar el total de participaciones (eventos) desde los hechos
-- TODO: Calcular el promedio general de 'score_numeric' con ROUND y casting a
::numeric
-- TODO: Calcular el score máximo con ROUND y casting a ::numeric
```

FROM ft_player_performance f

JOIN dim_persons_performance p ON -- TODO: Cruzar por llave de persona

GROUP BY -- TODO: Agrupar por nivel de membresía y género

ORDER BY -- TODO: Ordenar por membresía y el promedio general descendente

;

(5%) Consulta 3: Evaluar el impacto del estado del evento sobre las posiciones de juego. Esta consulta evalúa cómo el contexto operativo de los encuentros deportivos (por ejemplo, si el evento está programado, en curso o finalizado) afecta el rendimiento según la posición o rol que desempeña el participante en el terreno de juego. Al cruzar las posiciones con el estado del evento y calcular el puntaje promedio, la gerencia deportiva puede analizar si ciertas posiciones rinden mejor bajo determinadas condiciones del ciclo de vida del evento.

SQL

SELECT

```
-- TODO: Seleccionar la abreviatura de la posición (como position_code) y el
estado del evento
```

```
-- TODO: Contar el total de eventos desde la tabla de hechos
```

```
-- TODO: Calcular el promedio de 'score_numeric' para la posición/evento con
ROUND y ::numeric
```

FROM ft_player_performance f

JOIN dim_positions pos ON -- TODO: Cruzar la tabla de hechos con la dimensión de
posiciones



PONTIFICIA UNIVERSIDAD JAVERIANA CALI

Procesamiento de Grandes Volúmenes de Datos

Ingeniería de Sistemas y Computación

Departamento de Electrónica y Ciencias de la Computación

```
JOIN dim_events e ON -- TODO: Cruzar la tabla de hechos con la dimensión de eventos
GROUP BY -- TODO: Agrupar por abreviatura de posición y estado del evento
HAVING -- TODO: Filtrar para asegurar que el conteo de eventos sea mayor a 0
ORDER BY -- TODO: Ordenar descendentemente por el puntaje promedio de la posición
;
```

(7%) Consulta 4: Clasificar atletas por categoría utilizando DENSE_RANK. Esta consulta genera un listado competitivo clasificado de manera independiente dentro de cada nivel de membresía. Utilizando una función de ventana particionada por el tipo de membresía, ordena a los atletas según su puntaje promedio y les asigna un puesto de ranking (del 1 en adelante). Al filtrar el resultado para mostrar únicamente el Top 3 de cada categoría, el negocio puede premiar o destacar a los mejores deportistas de cada nivel sin que las categorías más altas eclipsen a las de menor jerarquía.

```
SQL
WITH athlete_scores AS (
    SELECT
        -- TODO: Seleccionar ID de persona, nombre completo (concatenando first_name
        y last_name) y nivel de membresía
        -- TODO: Calcular el promedio de 'score_numeric'
        -- TODO: Aplicar función DENSE_RANK() OVER (PARTITION BY ... ORDER BY ...
        DESC) para jerarquizar por membresía y promedio
    FROM ft_player_performance f
    JOIN dim_persons_performance p ON -- TODO: Cruzar por llave de persona
    GROUP BY -- TODO: Agrupar por id, nombre completo y membresía
)
SELECT
    -- TODO: Seleccionar nivel de membresía, nombre completo, promedio redondeado
    con ::numeric y el ranking obtenido
FROM athlete_scores
WHERE -- TODO: Filtrar para mostrar únicamente el Top 3 (ranking menor o igual a
3) por cada categoría
ORDER BY -- TODO: Ordenar por nivel de membresía y el ranking obtenido
```



PONTIFICIA UNIVERSIDAD JAVERIANA CALI

Procesamiento de Grandes Volúmenes de Datos

Ingeniería de Sistemas y Computación

Departamento de Electrónica y Ciencias de la Computación

;

(10%) Consulta 5: Analizar comparativamente el rendimiento individual frente al promedio de su categoría utilizando PERCENT_RANK. Esta consulta de nivel experto realiza una evaluación de desempeño relativo. Primero calcula el puntaje promedio global de cada nivel de membresía y lo compara con el promedio individual de cada atleta perteneciente a dicho nivel. Mide la desviación exacta (qué tan por encima o por debajo del promedio de su grupo se encuentra el deportista) y clasifica su posición porcentual mediante un percentil (PERCENT_RANK). Esto permite a los analistas identificar quiénes son verdaderos valores atípicos de alto rendimiento (outliers) en comparación directa con sus pares de la misma categoría.

```
SQL
WITH tier_stats AS (
    SELECT
        -- TODO: Seleccionar nivel de membresía y calcular el promedio general de
        'score_numeric' por categoría
        FROM ft_player_performance f
        JOIN dim_persons_performance p ON -- TODO: Cruzar por llave de persona
        GROUP BY -- TODO: Agrupar por nivel de membresía
),
participant_performance AS (
    SELECT
        -- TODO: Seleccionar ID, nombre completo (concatenado), membresía, conteo
        de eventos y promedio personal de score
        FROM ft_player_performance f
        JOIN dim_persons_performance p ON -- TODO: Cruzar por llave de persona
        GROUP BY -- TODO: Agrupar por id, nombre completo y membresía
)
SELECT
    -- TODO: Seleccionar nombre completo y nivel de membresía
    -- TODO: Calcular promedio personal redondeado con ::numeric
    -- TODO: Calcular promedio de la categoría (tier) redondeado con ::numeric
```



PONTIFICIA UNIVERSIDAD JAVERIANA CALI

Procesamiento de Grandes Volúmenes de Datos

Ingeniería de Sistemas y Computación

Departamento de Electrónica y Ciencias de la Computación

```
-- TODO: Calcular la diferencia numérica entre el promedio personal y el de la
categoría con su respectivo redondeo
-- TODO: Escribir un CASE que devuelva 'Por encima del promedio' o 'Por debajo
del promedio' según corresponda
-- TODO: Calcular el percentil dentro de la categoría usando PERCENT_RANK()
OVER (PARTITION BY ... ORDER BY ...) con redondeo a ::numeric
FROM participant_performance pp
JOIN tier_stats ts ON -- TODO: Cruza ambas CTEs utilizando el nivel de membresía
ORDER BY -- TODO: Ordenar por nivel de membresía y el promedio personal de forma
descendente
;
```